CC65 Programación concurrente y distribuida

Enunciado del Trabajo Final del Curso

Profesores:	Todos
Sección:	Todas
Fecha de evaluación:	Semana 15
Ciclo académico:	2026-20

Introducción:
El aprendizaje requiere un estímulo inicial que resulte interesante y nuevo. Precisamente el realizar un análisis del momento actual de la computación, resaltando la evolución de unidades de cómputo en dispositivos con gran capacidad para procesamiento paralelo y conectividad a redes. Así como, buscar la investigación por parte del estudiante de herramientas idóneas para el desarrollo de soluciones y el uso eficaz de los recursos computacionales.
En congruencia con ello, el trabajo final que se propone consiste en la construcción de una solución que implemente una aplicación de programación distribuida y Machine Learning con soporte de Apis, para el problema planteado. El equipo de trabajo utilizará Github como herramienta colaborativa para la gestión del desarrollo de software y Docker compose para el despliegue de la solución.

Objetivo:
El presente documento define el trabajo final y la rúbrica que permite evaluar el logro del curso CC65 Programación Concurrente y Distribuida. El objetivo del trabajo final (TF) es que los estudiantes construyan aplicaciones concurrentes y distribuidas de alto rendimiento de manera eficaz desde el punto de vista de la ingeniería de software.

Logro del curso:

Al finalizar el curso, el estudiante construye aplicaciones concurrentes y distribuidas de alto rendimiento de manera eficaz.
El curso busca desarrollar la competencia student outcome 4: La capacidad de reconocer responsabilidades éticas y profesionales en situaciones de ingeniería y hacer juicios informados, que deben considerar el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales, para Ciencias de computación e Ingeniería de Software.

Instrucciones:

- El trabajo es en equipos de mínimo 2 y máximo 3 estudiantes, se recomienda mantener los grupos formados para el trabajo parcial.
- El trabajo está dividido en 3 entregables.
- Se usará software para detección de plagio.
- El proyecto se aloja en Github y debe seguir Git Flow. Repositorio público.
- Seguimiento de tareas del proyecto usando tablero Kanban en Github.
- El trabajo final será desarrollado íntegramente en GO y contenerizado en Docker.
- Utilizará Docker compose para desplegar la solución, UI, Server API, Cluster Nodos ML, Bases de datos.

Alcance del trabajo:

1. Objetivo

El trabajo combina dos grandes habilidades que has aprendido durante el curso: procesar información de forma paralela (varios procesos trabajando al mismo tiempo) y distribuida (varios computadores cooperando) de alto rendimiento de manera eficaz, procesando grandes volúmenes de datos reales del Portal de Datos Abiertos del Gobierno del Perú (datosabiertos.gob.pe), aplicadas a un problema real que beneficia a personas reales.

No se trata solo de escribir código que funcione. Se trata de construir un sistema que sea rápido, robusto y útil, algo de lo que puedas estar orgulloso al mostrarlo fuera del aula.

2. Alcance del proyecto

Cada equipo tendrá asignado uno de los 10 conjuntos de datos sugeridos del Portal de Datos Abiertos del Perú. La solución deberá:

- ✓ Cargar y procesar más de 1,000,000 de registros de forma concurrente.
- ✓ Entrenar o ejecutar un modelo de Machine Learning Masivo (MML) de manera distribuida a través de un clúster de nodos (usando Random Forest Distribuido, K-Means Paralelo, Logistic Regression Distribuida o Neural Networks con Data Parallelism).
- ✓ Exponer una API REST/WebSocket Coordinator que gestione la comunicación TCP con los nodos de cómputo y persista métricas en MongoDB y Redis.
- ✓ Desplegar una Interfaz Web (SPA) que exponga las predicciones en tiempo real y visualice el impacto social de la solución.

3. Algoritmos MML

A continuación, se valida técnicamente la aplicación de los 4 algoritmos MML sugeridos sobre datasets masivos en un entorno distribuido en Go:

- ✓ Random Forest Distribuido (Clasificación masiva / Regresión en nodos paralelos):

- Mecanismo: Cada nodo del clúster entrena un subconjunto de árboles de decisión (Data Parallelism / Bagging) sobre un fragmento del dataset (sharding). Los nodos votan o promedian la predicción final en el nodo coordinador (API).

- Aplicabilidad: Excelente para clasificación/regresión multivariable con más de 1M de registros.

✓ K-Means Paralelo / Distribuido (Clustering de grandes volúmenes de datos):

- Mecanismo: MapReduce o K-Means. Cada nodo calcula las distancias de su partición de datos local a los centroides globales y acumula sumas parciales. La API/Master consolida y actualiza los centroides en cada iteración.

- Aplicabilidad: Altamente factible para segmentación geográfica, detección de anomalías y patrones socioeconómicos.

✓ Logistic Regression Distribuida (Clasificación binaria masiva):

- Mecanismo: Descenso de Gradiente Distribuido (SGD o Parameter Server/AllReduce). Cada nodo procesa un lote (mini-batch) de datos, calcula el gradiente del error local y transmite los gradientes vía TCP al coordinador para actualizar los pesos del modelo.

- Aplicabilidad: Ideal para predecir eventos binarios (ej. riesgo sí/no, fraude sí/no, severidad alta/baja).

✓ Neural Networks con Data Parallelism (Entrenamiento distribuido):

- Mecanismo: División del dataset en mini-batches distribuidos entre los nodos. Cada nodo ejecuta el paso forward y backward localmente, enviando los gradientes a la API para la sincronización de pesos globales.

- Aplicabilidad: Viable utilizando arquitecturas Feedforward / Perceptrón Multicapa (MLP) en Go para problemas complejos de clasificación/regresión.

4. El problema que vas a resolver

N°	Caso / Proyecto Sugerido	Dataset Base (Portal de Datos Abiertos Perú)	Volumen Est.	Algoritmo MML Recomendado	Tipo de Problema y Planteamiento del Proyecto
1	**Predicción de Riesgo Epidemiológico Nacional (COVID-19 / Dengue)**	Casos positivos acumulados y fallecidos (MINSA - OGDN)	>2.5M filas	**Logistic Regression Distribuida**	Clasificación binaria para predecir la probabilidad de severidad / hospitalización del paciente según comorbilidades y geolocalización.
2	**Análisis y Clasificación de Delitos y Faltas Urbanas**	Denuncias Policiales registradas (IN EI / MININTER - PNP)	>1.8M filas	**Random Forest Distribuido**	Clasificación multiclase del tipo de delito según horario, zona urbana y patrón criminológico para patrullaje preventivo.
3	**Zonificación y Riesgo de Accidentes de Tránsito**	Ocurrencia de Accidentes de Tránsito Nacionales (SUTRAN / MTC)	>1.2M filas	**K-Means Paralelo**	Clustering geospacial masivo para identificar "puntos negros" (zonas de alta siniestralidad) y coordinar ambulancias/fiscalización.
4	**Predicción de Contrataciones y Riesgo en Compras Públicas**	Órdenes de Compra y Servicio del Estado (OSCE / MEF)	>3.0M filas	**Random Forest / Logistic Reg.**	Clasificación binaria para detectar compras con alto riesgo de sobrecosto o cancelación previa a la adjudicación.
5	**Monitoreo y Predicción de Consumo Eléctrico Doméstico**	Facturación y consumo de energía por cliente (Osinergmin / Empresas Eléctricas)	>4.0M filas	**Neural Networks (Data Parallelism)**	Regresión/Clasificación multivariable para estimar picos de demanda y prevenir colapsos en la red eléctrica regional.
6	**Análisis Masivo de la Ejecución del Presupuesto Público**	Consulta Amigable de Ejecución Gasto e Ingresos (MEF)	>2.0M filas	**K-Means Paralelo**	Clustering socioeconómico de municipalidades para detectar desviación atípica en la ejecución presupuestal de obras públicas.
7	**Clasificación y Seguimiento de la Vacunación Nacional**	Registro Nacional de Vacunación (HIS-MINSA / REUNIS)	>5.0M filas	**Logistic Regression Distribuida**	Clasificación binaria para predecir la tasa de deserción en la aplicación de segundas/terceras dosis en zonas rurales.
8	**Estimación de Precios de Venta e Inmuebles Agrícolas / Urbanos**	Padrón de Productores Agrícolas y Propiedad Inmueble (MIDAGRI / SUNARP)	>1.5M filas	**Neural Networks (Data Parallelism)**	Regresión paralela para estimar el valor del suelo agrícola en función de la disponibilidad hídrica, altitud y conectividad.
9	**Evaluación de Rendimiento y Deserción Escolar**	Evaluaciones Censales de Estudiantes (ECE - MINEDU)	>1.1M filas	**Random Forest Distribuido**	Clasificación multivariable del nivel de logro educativo e identificación temprana de colegios en riesgo de alta deserción.
10	**Monitoreo Calidad del Aire y Sensores Ambientales**	Lecturas continuas de Estaciones Meteorológicas y Calidad del Aire (SENAMHI / OEFA)	>3.5M filas	**K-Means / Neural Networks**	Clustering y predicción masiva de concentraciones de PM2.5/PM10 para alertas tempranas de salud respiratoria.

Tu equipo elegirá uno de estos casos (coordinará con el profesor para no repetir los casos con otros grupos)

El impacto social no es un requisito vago. Significa que el sistema que construyas debe responder al menos una pregunta que importe a personas fuera de la universidad. Algunos ejemplos concretos:

- ¿En qué horas y zonas hay mayor riesgo de accidente de tránsito para planificar patrullaje preventivo?

- ¿Cuándo se disparará la contaminación del aire para alertar a personas con enfermedades respiratorias?

- ¿En qué hospitales habrá escasez de camas en los próximos días para redistribuir recursos médicos?

Si tu sistema puede responder algo así, tiene impacto social. Esa pregunta debe quedar documentada desde el inicio del proyecto.

5. Visión de Arquitectura

El sistema tiene cuatro partes bien definidas que funcionan como una cadena. Cada parte puede desarrollarse de forma independiente organizado por los miembros del equipo, lo que permite trabajar en paralelo (igual que el código que van a escribir).

Parte A — Cargador de datos concurrente

Es la entrada del sistema. Su trabajo es leer el archivo de datos (que puede tener millones de filas) y procesarlo en paralelo. En lugar de leer línea por línea de forma secuencial, divide el archivo en bloques y asigna cada bloque a un worker diferente.

En lenguaje Go esto se hace con goroutines y channels. El resultado de esta etapa es un conjunto de datos limpio, validado y listo para entrenar.

Parte B — Motor de entrenamiento distribuido

Es la parte más exigente del proyecto. Toma los datos limpios y entrena un modelo de aprendizaje automático usando múltiples hilos o procesos. El tipo de modelo puede ser simple, como una regresión logística o un árbol de decisión, lo importante no es la complejidad del modelo sino que el entrenamiento ocurra de forma genuinamente paralela.

En lenguaje Go esto se hace con Sockets y patrones. Aquí se demostrará el conocimiento central del curso: cómo dividir trabajo, cómo sincronizar resultados parciales y cómo garantizar que no haya condiciones de carrera en la actualización de los parámetros del modelo.

Parte C — API de predicciones

Una vez entrenado el modelo, esta parte lo expone como un servicio web. Cualquier persona puede enviar una consulta (por ejemplo, la hora y ubicación de un viaje) y recibir una predicción (por ejemplo, la probabilidad de demora). La API escrita en lenguaje Go usando funciones de Web Server HTTP debe responder en menos de 100 milisegundos para ser considerada aceptable (evidenciará en las pruebas dentro del informe final).

Parte D — Visualización de impacto

Es la parte más orientada a comunicar los resultados. Puede ser una interfaz SPA, use algún framework por ejemplo: Angular, React con componentes simples o un dashboard con gráficos y métricas. Su función es mostrar, con datos reales, qué problema resuelve el sistema y a cuántas personas podría beneficiar.

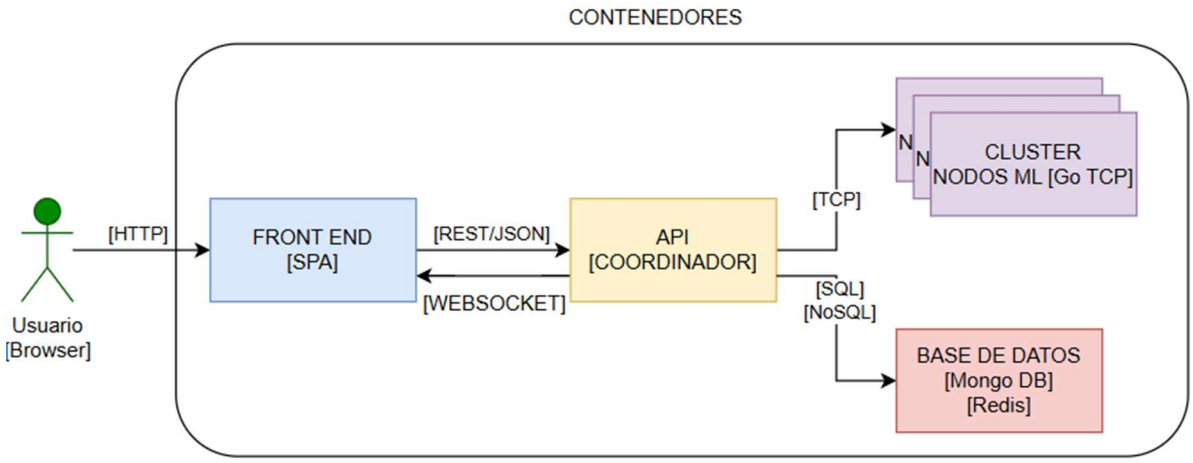


Fig1. Vista de Arquitectura de Referencia

6. Detalle de la arquitectura

✓ Clúster Nodos (ML):

- Implementa el algoritmo concurrente/distribuido.
- Procesa grandes volúmenes de datos
- Usa goroutines y channels (Go) para dividir y combinar cálculos.
- Comunica resultados al coordinador (API) mediante TCP interno.

✓ API:

- Implementada en Go (recomendado para continuidad con la lógica concurrente).
- Expone endpoints.
- Puede gestionar la autenticación con JWT tokens.
- Hacer de coordinador del cluster, enviando tareas a los nodos de cómputo y unificando resultados.
- Almacenar resultados de recomendaciones en base de datos.
- Servir datos en tiempo real al frontend en formato JSON usando websockets

✓ Base de datos:

- Incluir dos niveles de almacenamiento:
  - Persistente: MongoDB.
  - Cache/colaborativo: Redis: Recomendaciones precalculadas o respuestas parciales.

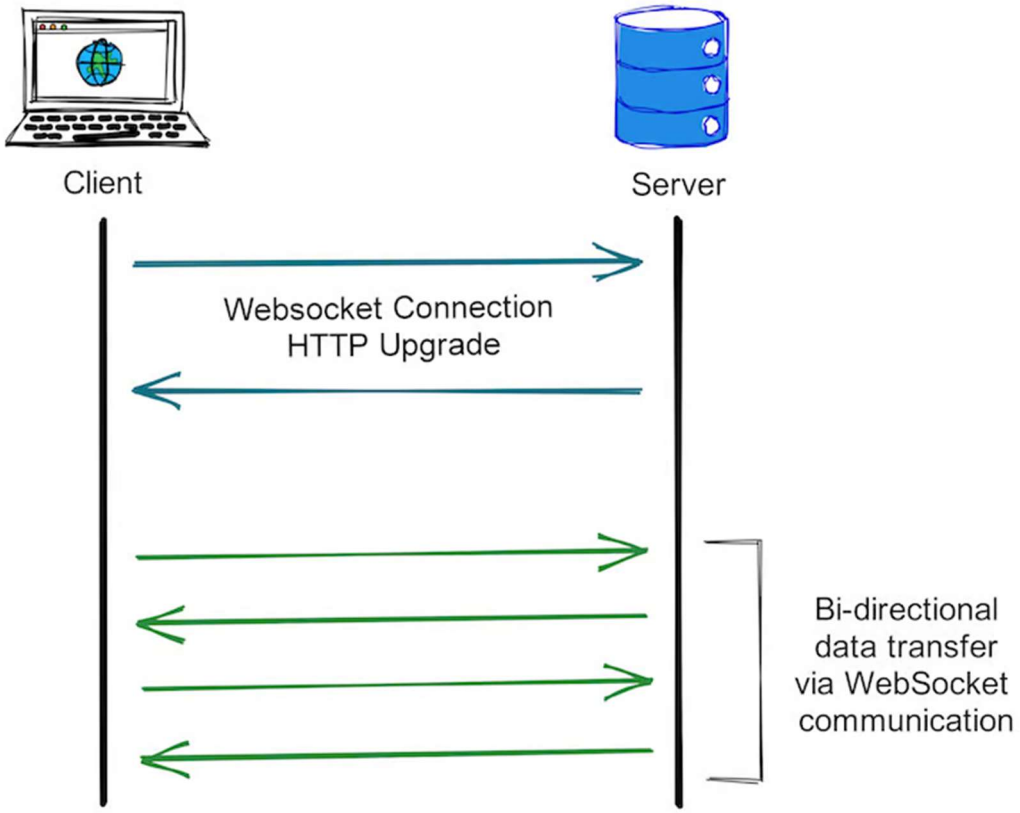
✓ FrontEnd Web:

- Desarrollar interfaz SPA, use algún framework por ejemplo: Angular, React con componentes simples
- Módulos:
  - Inicio de sesión / autenticación.
  - Panel de usuario (Dashboard).
  - Visualización de resultados (entrenamiento y simulación)
  - Panel administrador: métricas del cluster (uso CPU, latencia, número de nodos, logs).

7. Anexos

La especificación del protocolo WebSocket define dos esquemas URI:

- WebSocket (ws): used for non-encrypted connections
- WebSocket Secure (wss): used for encrypted connections



Evaluación del Trabajo Final

Instrucciones de Entrega:

- Código fuente de la solución eliminando cualquier defecto, con datos de prueba, evidencia de pruebas realizadas.
- Incluir las imágenes y demás recursos utilizados para la elaboración de la solución y pruebas.
- Video de presentación de la solución, puntos importantes de su construcción y su funcionamiento (end to end), realizado en un máximo de 6 minutos donde participen todos los integrantes y demuestren el conocimiento del tema.
- El trabajo será entregado por cada integrante del grupo y mediante el aula virtual.
- El plazo es impostergable y por ningún motivo y/o circunstancia se recibirá trabajos fuera de esa fecha y hora, ni por otro medio diferente al aula virtual.

Plazos de Entrega:

- Entregable 1(PC3) – Fecha de entrega semana 11 (07/11/2026 23:30 Hrs.)
  - Elaborar informe de los puntos:
    - Presentación del caso a resolver (Problema y motivación, pregunta que atiende el impacto social)
    - Limpieza y Análisis de datos (procedimiento y evidencias)
    - Diseño del modelo ML distribuido (explicación del algoritmo)
    - Paralelización del cálculo (justificación del planteamiento, técnicas y patrones usados en lenguaje go)
    - Evidencias de la implementación (elaboración de código en go y pruebas)
    - Suben sus cambios a github
    - Actualizan el tablero Kanban de proyecto en github
    - Elabora el Reporte de participación
- Entregable 2(PC4) – Fecha de entrega semana 13 (21/11/2026 23:30 Hrs.)
  - Elaborar informe de los puntos:
    - Distribución (Cluster de nodos ML, justificación de topología de red, explicar el funcionamiento en lenguaje go)
    - Desarrollo de API (swagger, explicar el funcionamiento en lenguaje go, debe responder en menos de 100 milisegundos de latencia capturar pantallazos de pruebas con postman)
    - Implementación de Bases de datos (diseño de entidades, registros)
    - Evidencias de la implementación y pruebas de funcionamiento (datos de prueba y pantallazos)
    - Documentación (el informe debe contener el entregable 3 con las correcciones)
    - Suben sus cambios a github
    - Actualizan el tablero Kanban de proyecto en github
    - Elabora el Reporte de participación
- Entregable 3(TB2) – Fecha de entrega semana 14 (29/11/2026 23:30 Hrs.)
  - Elaborar informe de los puntos:

- Desarrollo FrontEnd Web (Diseño adecuado a UX para usuario final)
- Integrar el frontend con el API concentradora (configuración de políticas de seguridad de API)
- Evaluación experimental (datos de prueba, evidencia de pruebas de funcionalidad, responde a preguntas que atiende el impacto social)
- Documentación y Presentación Final (el informe final contiene los entregables 3 y 4 con las correcciones)
- Suben sus cambios a github
- Actualizan el tablero Kanban de proyecto en github
- Elabora el Reporte de participación

Presentación y sustentación de trabajo final en la sesión de clase de la semana 15, cada grupo es responsable de presentar el funcionamiento del sistema y responder a las preguntas del docente para su evaluación.

Detalle de Entrega:

1. El informe del trabajo final debe ser en formato Word y tener el nombre con el siguiente formato: 1ACC0065_YYY_202610_Informe_UXXXXXXXX, en donde los caracteres YYY reemplaza el código de entregable (PC3, PC4, TB2), las X se reemplazarán por el código de alumno.
2. Puede subir archivos de código adicional al informe en archivo ZIP o RAR y tendrá por nombre: 1ACC0065_YYY_UXXXXXXXXX.
3. Solo se calificarán los trabajos entregados mediante el Aula virtual.
4. Estamos seguros de que cada uno realizará su trabajo, sin embargo, para evitar cualquier perspicacia, le recomendamos leer el reglamento de disciplina del alumno, en el cual se indican las faltas y las sanciones que se indican en el caso de haber copia de trabajos.

Los informes de los entregables deberán tener la siguiente estructura:

a) Carátula
b) Resumen
c) Índice
d) Descripción del problema y motivación
e) Objetivos
f) Desarrollo (Evidencias de la implementación de cada etapa y arquitectura)
g) Conclusiones
h) Recomendaciones
i) Bibliografía (utilizar APA7)
j) Anexos (Link de repositorio GitHub, link de video, documentos, informes, otros)

Links de ayuda

Tablero Kanban en GitHub:

- https://youtu.be/Vjf_s7TGmqY

Rúbrica

	Sobresaliente	En Proceso	Deficiente
Planteamiento	3 puntos Definición del problema, dataset elegido. Usa diagrama de arquitectura solución detallada para complementar la explicación de su planteamiento solución. Programación concurrente y distribuida. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	2 puntos Define procesos e Información incompleta. Expone y no demuestra conocimiento total del tema.	0 puntos No elaborado
Implementación	8 puntos Carga, limpieza y procesamiento concurrente de >1M de registros. Entrenamiento paralelo del modelo ML usando goroutines. Funciona correctamente, utiliza docker, puertos, para procesar los datos generados por la solución concurrentemente, usa canales y muestra resultados siguiendo las consideraciones de la arquitectura solución. Usa Apis para exponer métodos que consuma el front end	5 puntos Elabora el código y ejecuta concurrentemente, usa puertos, canales, muestra resultados, implementando las consideraciones del trabajo de forma parcial. Expone y demuestra conocimiento parcial del tema.	0 puntos No elaborado

	y recopile los resultados del cluster. Servicio REST con predicciones en tiempo real y métricas de rendimiento. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.		
Interfaz	5 puntos La interfaz en modo gráfico usando un framework UI, muestra un menú de opciones, configura el aplicativo con parámetros ingresados desde la UI, muestra los resultados en tiempo real, demuestra una adecuada experiencia de usuario y organizada en página SPA que muestre resultados e impacto social. Tiene en cuenta las restricciones del trabajo. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	3 puntos Muestra el funcionamiento de la aplicación y resultados entendible en página web parcialmente. Expone y demuestra conocimiento parcial del tema.	0 puntos No elaborado
Informe	2 puntos Hace buen uso del medio escrito, cumple con la estructura del informe, guarda coherencia y presenta resultados, conclusiones entendibles y orientadas a resultados. Anexa el	1 puntos Sigue una estructura clara y contiene los elementos mínimos necesarios solicitados. Usa herramienta de versionado de código. Informe incompleto. Expone y demuestra	0 puntos No elaborado, No demuestra conocimiento del tema.

	historial del versionamiento del código fuente, evidencia el git Flow. Demuestra el correcto seguimiento de las actividades del proyecto en el tablero Kanban de github. Cumple con el reporte de participación. Cumple con la exposición de su trabajo demostrando conocimiento total del tema.	conocimiento parcial del tema.	
Video	2 puntos Elabora un video de 6 minutos como máximo donde el grupo presenta la solución, puntos importantes de su construcción y su funcionalidad (end to end), demostrando dominio del tema.	1 puntos Cubre los puntos solicitados de forma parcial.	0 puntos No elaborar el video afecta la calificación en los demás ítems de la rúbrica.

Santiago de Surco, octubre de 2026